

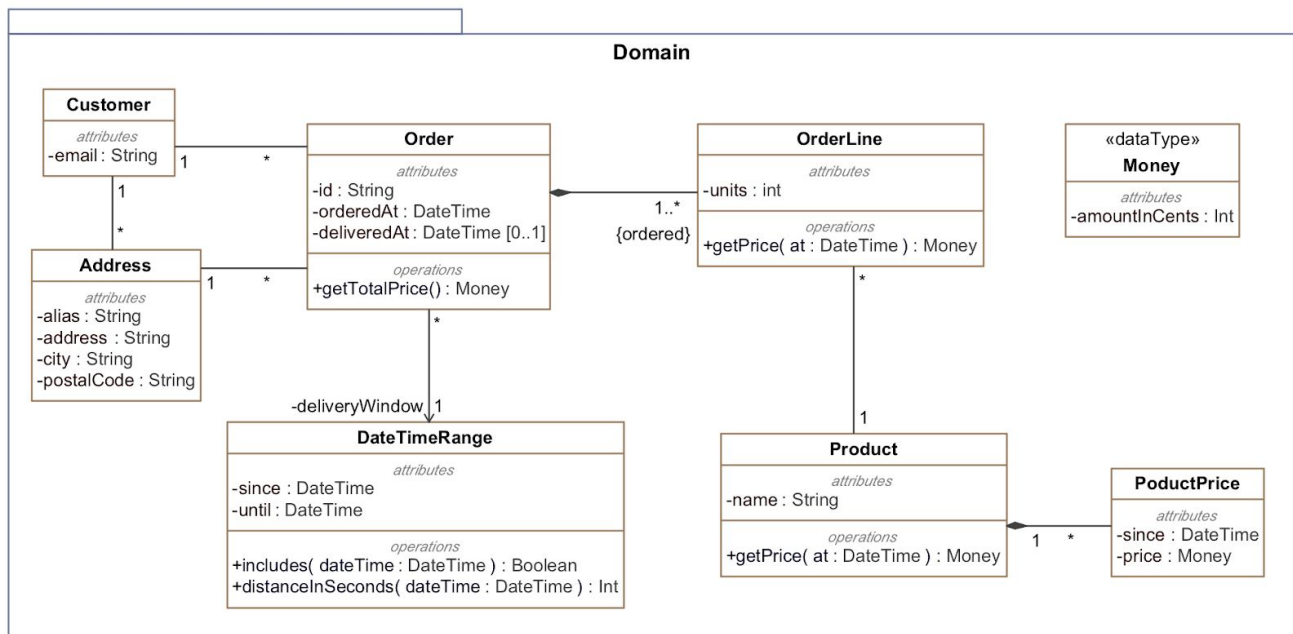
Análisis y Diseño con Patrones

Práctica 2: Principios de asignación de responsabilidades y de diseño

Para poder elaborar esta práctica partiremos de la solución de la Práctica 1.

Recuerda que nos habían contratado para desarrollar un software para gestionar pedidos de compras online. Como en la solución publicada de la práctica 1 había varias variantes, mostramos aquí el diseño de partida, en el que hemos hecho algunos ajustes menores (comentados en el pie de los diagramas).

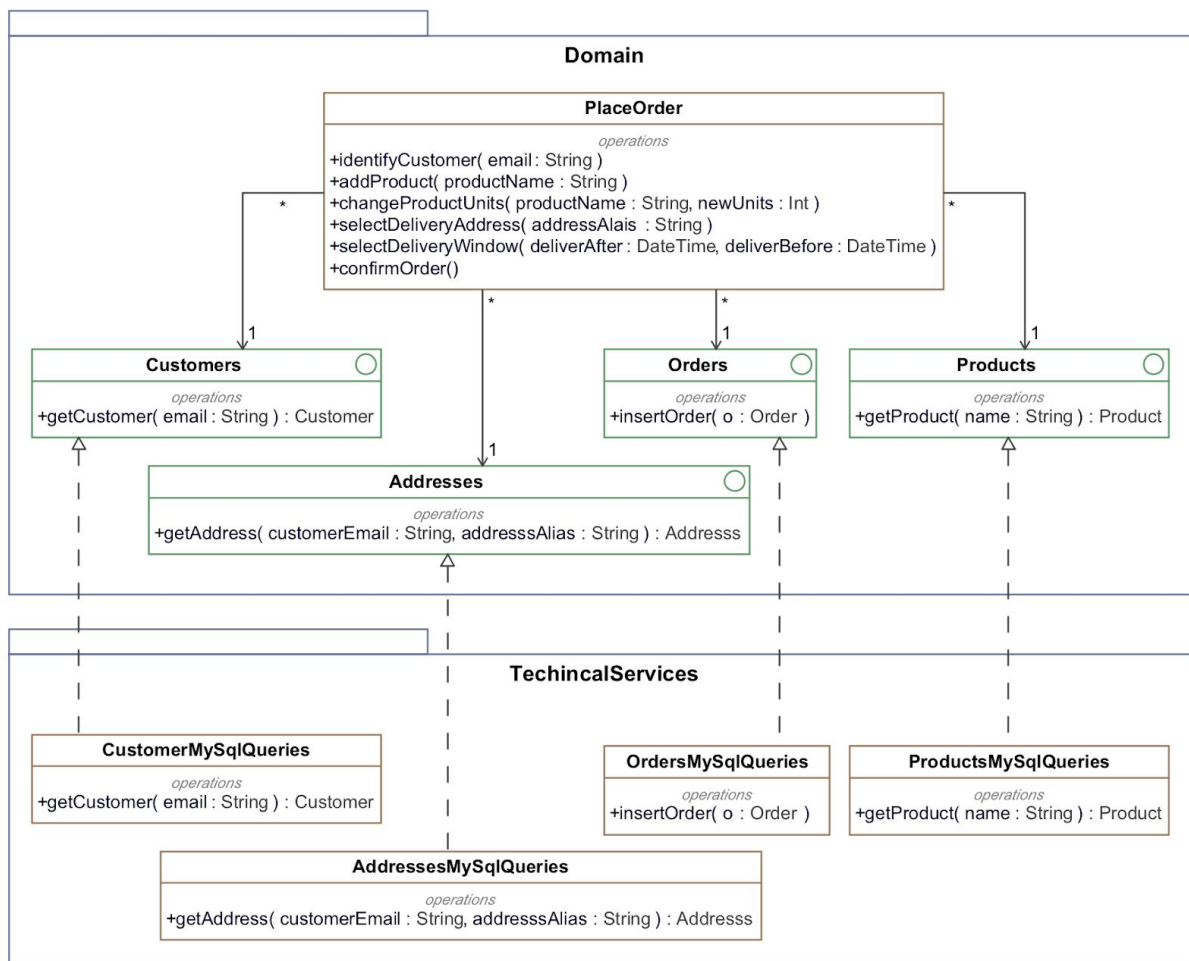
El modelo del dominio de partida es éste:



- El nombre de clase *Currency* causaba confusión. Lo hemos sustituido por *Money*. Una instancia de *Money* representa un precio, una cantidad de dinero como por ejemplo 3,14 €. Lo hemos representado como un *data type* con un único campo que representa la cantidad en céntimos, asumiendo que no necesitamos soportar más de una moneda.

- El diseño del que partimos ya aplicaba el patrón Arquitectura en Capas. Hemos representado explícitamente que todas las clases que se muestran en el diagrama son de la capa de dominio. Para ello hemos moldeado la capa de dominio como un *package* UML.
- Hemos integrado la solución del ejercicio 1 de la Práctica 1 y, por tanto, cuando pedimos el precio a un producto o a una línea de pedido, hay que indicar en qué fecha queremos saber el precio.

PlaceOrder y las clases que utiliza tienen este diseño:



- Hemos representado el hecho de que *PlaceOrder* tiene un atributo de tipo *Customers* como una asociación dirigida con multiplicidad 1 junto a *Customers*. Hemos hecho lo mismo para *Addresses*, *Orders* y *Products*.
- Hemos renombrado la clase *CommandsMySQLQueries* por *OrdersMySQLQueries* por coherencia con la terminología de *Orders*.

- De nuevo, hemos representado la pertenencia a capas mediante la pertenencia a *packages* UML.

Comentario previo a la solución de la práctica

En el enunciado de esta práctica se pedían soluciones en pseudocódigo orientado a objetos o en diagramas UML. En esta solución hemos elegido para utilizar el lenguaje de programación Java como pseudocódigo orientado a objetos, ya que es uno de los lenguajes orientado a objetos más popular. Pero naturalmente, cualquier lenguaje o pseudocódigo orientado a objetos que muestre el diseño que se propone se considerará válido.

Ejercicio 1 (20%)

Nos fijaremos ahora en la clase *PlaceOrder* que ya conocemos. Queremos diseñar el detalle de las operaciones *addProduct(productName: String)* y *changeProductUnits(productName: String, newUnits: Int)*.

La primera operación (*addProduct*) debe comportarse de manera que:

- Si el pedido en preparación (*this*) ya tiene un producto con el mismo nombre (en *this.orderLines*), se incrementa en 1 las unidades de este producto.
- Si el pedido en preparación (*this*) no tiene un producto con el mismo nombre, se crea una nueva instancia de *OrderLine* con el producto indicado y 1 unidad y se añade al final de *this.orderLines*.

La segunda (*changeProductUnits*) debe comportarse de manera que:

- Si el pedido en preparación (*this*) ya tiene un producto con el mismo nombre (en *this.orderLines*), las unidades del producto de aquella línea pasan a ser *newUnits*.
- Si el pedido en preparación (*this*) no tiene un producto con el mismo nombre, se crea una nueva instancia de *OrderLine* con el producto indicado y con *newUnits* unidades.

El objetivo es rediseñar el primer esbozo que teníamos de la operación *addProduct* (ya que no hacía todo lo que aquí se describe) y diseñar también la operación *changeProductUnits*.

- a) Diseña en pseudocódigo o en un diagrama de secuencias UML las operaciones *addProduct(productName: String)* y *changeProductUnits(productName: String, newUnits: Interés Legislación)* teniendo en cuenta que:
 - Hay que asignar responsabilidades de forma apropiada.
 - Hay que evitar la repetición de código (principio de No Repetición o DRY).
- b) Justifica tu solución razonando sobre cómo has asignado las diferentes responsabilidades a las clases.

Nota:

- Tu solución debe ser coherente con el diseño inicial presentado en este enunciado.

Solución:

Mostramos primero el código resultante:

```
public class PlaceOrder {

    private List<OrderLine> orderLines = new LinkedList<>();

    ...

    private OrderLine findOrderLine(String productName){
        boolean found = false;
        for(OrderLine ol : orderLines) {
            if (ol.isForProduct(productName)) {
                return ol;
            }
        }
        return null;
    }

    public void addProduct(String productName) {
        OrderLine orderLine = findOrderLine(productName);
        if(orderLine == null){
            Product p = products.getProduct(productName);
            orderLines.add(new OrderLine(p, 1));
        } else {
```

```

        orderLine.addOne();
    }
}

public void changeProductUnits(String productName, int newUnits){
    OrderLine orderLine = findOrderLine(productName);
    if(orderLine == null){
        throw new RuntimeException("Order line not found");
    }
    orderLine.setAmount(newUnits);
}

}

public class OrderLine {

    ...

    public boolean isForProduct(String productName) {
        return product.getName().equals(productName);
    }

    public void addOne() {
        this.units += 1;
    }

    public void setAmount(int newUnits) {
        this.units = newUnits;
    }
}

```

Hablemos primero sobre la operación *addProduct*:

Como podemos ver, primero debe comprobar si hay o no una línea de pedido para ese producto. Hemos creado una función aparte para la búsqueda de esta línea porque más adelante hemos encontrado que también la teníamos que usar en *changeProductUnits* y así honramos el **principio de No Repetición**.

La búsqueda no tiene demasiada complicación. La construcción *for* del código Java utiliza internamente un iterador; este tipo de construcción *for* tiene versiones similares en otros lenguajes de programación y también usan internamente iteradores. Pero técnicamente hemos utilizado una solución ya implementada y basada en iterador, más que haber aplicado el patrón iterador para diseñar una solución.

Por otra parte, hemos aplicado el **patrón Experto** y en lugar de pedir el nombre del producto a la línea para compararlo con el que nosotros buscamos hemos pedido a la propia línea si *isForProduct(productName)*.

Continuando con *addProduct*, si se ha encontrado una línea para ese producto, es necesario incrementar la cantidad de la línea en 1. Para ello, el **patrón Experto** nos dice que debe ser el propio objeto de tipo *OrderLine* el que haga este incremento.

En caso de que no se haya encontrado ninguna línea, hay que crear una y añadirla a *this.orderLines*. Según el **patrón Creador**, el propio objeto de clase *PlaceOrder* es un buen creador, ya que agrega líneas de pedido.

En cuanto a la operación *changeProductUnits*:

Una vez detectado que no queríamos repetir el código de búsqueda de la línea, hemos utilizado la misma funcionalidad de búsqueda de una línea. Podíamos asumir que la línea buscada existiría, pero hemos decidido lanzar una excepción en caso de que no existiera, para mayor seguridad.

Una vez encontrada la línea, según el **patrón Experto**, la encargada de modificar la cantidad es la propia línea. No se trata, pues, de utilizar lo que en Java llaman un *setter* sino de detectar que necesitamos una operación para modificar la cantidad de una línea.

Cancelación de líneas de pedido

Ahora nos fijaremos en un nuevo caso de uso que permite a los usuarios modificar (de forma restringida) sus pedidos una vez los han realizado. Para simplificar el enunciado, nos centraremos

únicamente en la posibilidad de cancelar una línea del pedido (eliminar del pedido un producto, con todas sus unidades).

Cuando un usuario hace un pedido, éste queda a la espera para ser preparado y enviado. En este estado, un usuario puede pedir cancelar cualquier línea de pedido y la cancelación tiene un efecto inmediato, como si nunca hubiera pedido ese producto.

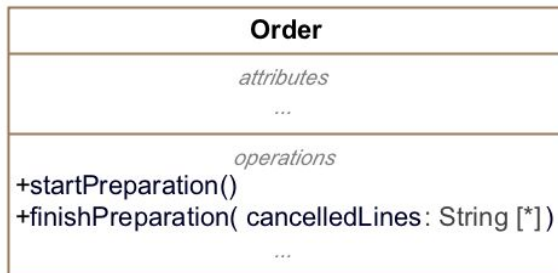
La preparación del pedido se inicia cuando uno de los empleados encargados de preparar pedidos lo selecciona para su preparación. A partir de ese momento el pedido está en preparación; en este estado el usuario todavía puede solicitar la cancelación de líneas de pedido, pero esta cancelación se guarda como pendiente de confirmación. El empleado que hace la preparación decidirá si puede cancelar o no aquellas líneas con solicitud de cancelación. Esta persona, al finalizar la preparación del pedido, lo indicará al sistema y, en caso de que hubiera solicitudes de líneas de cancelación, indicará qué líneas han sido finalmente canceladas.

Cuando un pedido ya está preparado (se ha finalizado su preparación), está listo para su envío y cualquier solicitud de cancelar líneas queda automáticamente denegada.

Ejercicio 2 (20%)

Como hemos visto, en función de si un pedido se ha creado, está en preparación o está preparado, el comportamiento de la solicitud de cancelación cambia. Más adelante tendremos que diseñar otras funcionalidades que se comportarán de forma diferente en función de si el pedido ha sido enviado o no y de si ha sido recibido por el cliente o no, y queremos seguir el principio abierto / cerrado con respecto a estas extensiones.

Supón que eventualmente, por patrón Experto, la clase *Order* dispondrá de las siguientes operaciones necesarias para indicar que el pedido se ha empezado a preparar o que su preparación ha finalizado:



```
public class Order {
    ...
    void startPreparation(){...}
    void finishPreparation(List<String> cancelledLines){...}
}
```

Hay un patrón de diseño que nos permite resolver exactamente esta problemática.

- Diseña en pseudocódigo o en un diagrama de secuencias UML las operaciones *startPreparation()* y *finishPreparation(List<String> cancelledLines)* teniendo en cuenta las necesidades descritas y que hay que asignar responsabilidades de forma apropiada.
- Justifica tu solución razonando sobre qué patrón de diseño has aplicado y por qué es adecuado.

Notas:

- Supón que estas operaciones deben fallar en caso de que no sean posibles. Así, si intentamos comenzar la preparación de un pedido que ya está preparado o en preparación, la operación debe fallar. Supón que puedes lanzar una excepción como se hace en los lenguajes de programación actuales. Para ello puedes utilizar con un comentario en el diagrama UML o una sentencia *throws Exception()* en tu pseudocódigo.
- La solución que propongamos debe ser coherente con el diseño inicial presentado en este enunciado y con tu solución del ejercicio anterior.

Solución:

En este caso podemos ver que el comportamiento de un pedido depende de su estado. Podríamos haber diseñado este comportamiento sencillamente poniendo condicionales donde sea necesario

para comprobar el estado. Pero el **patrón Estado** es adecuado en este caso y nos permite diseñar un modelo más extensible que haga evidentes las transiciones entre estados.

Tal como describe el propio patrón hemos tenido que tener en cuenta varias consideraciones:

- Hemos decidido que las transiciones entre estados las coordine el estado, ya que en función del estado actual serán válidas unas u otras transiciones.
- Hemos decidido que los objetos que representan los estados no serían compartidos entre contextos, ya que nos ha parecido una solución más sencilla y natural.
- Por lo tanto, hemos asumido que al cambiar de estado creamos un nuevo objeto que representa el nuevo estado. En ese mismo momento podríamos destruir el objeto que representa el estado anterior si en la plataforma de ejecución (en este caso, Java) fuera necesario liberar memoria a mano.

Veamos, pues, la clase `Order`:

```
public class Order {
    ...

    private OrderState state;

    void startPreparation(){
        this.state.startPreparation();
    }

    void finishPreparation(List<String> cancelledLines){
        this.state.finishPreparation(cancelledLines);
    }
}
```

Como podemos ver, tanto *startPreparation* como *finishPreparation* sólo delegan en el estado del objeto, ya que el comportamiento de ambas cambia en función del estado del pedido.

OrderState es una clase abstracta que siempre asume que la operación solicitada no es posible porque el estado no es el correcto. Tal y como indicaba el enunciado, hemos decidido que en caso de que se invoque una operación que no es posible debido al estado actual se lanzaría una excepción:

```
public abstract class OrderState {

    void startPreparation() {
        throw new IllegalStateException();
    }

    void finishPreparation(List<String> cancelledLines) {
        throw new IllegalStateException();
    }

}
```

Hemos llamado *PlacedOrderState* al estado que tiene un pedido que se acaba de crear y del que no se ha iniciado aún la preparación. En tal estado se puede hacer *startPreparation*. Esta operación sólo debe cambiar el estado del pedido. Pero para ello necesitamos el pedido. Hemos asumido, pues, que al construir una instancia de *PlacedOrderState* nos pasan el pedido al que pertenece, de tal manera que esta instancia pueda invocar una operación *setState* sobre el pedido para cambiar su estado.

```
public class PlacedOrderState extends OrderState {
    private final Order order;

    PlacedOrderState(Order order) {
        this.order = order;
    }

    @Override
    public void startPreparation() {
        order.setState(new InPreparationOrderState(order));
    }

}
```

En *Order*, pues, hemos tenido que añadir una operación para cambiar el estado del pedido. Esta operación sólo se invocará desde un estado cuando éste decida que hay que cambiar el estado del pedido.

```
public class Order {
    ...
    void setState(OrderState state) {
        this.state = state;
    }
}
```

```

    }
}

```

Hemos llamado *InPreparationOrderState* al estado de los pedidos en preparación. En este estado es posible invocar *finishPreparation*. El funcionamiento es similar al caso anterior en el sentido de que hay que cambiar el estado del pedido. Pero en este caso, además, hay que borrar las líneas que nos indiquen como canceladas.

```

public class InPreparationOrderState extends OrderState {

    private final Order order;

    InPreparationOrderState(Order order) {
        this.order = order;
    }

    @Override
    public void finishPreparation(List<String> cancelledLines) {
        this.order.removeLines(cancelledLines);
        this.order.setState(new PreparedOrderState(order));
    }

}

```

Aunque este diseño es bastante simétrico al de la operación anterior (porque utiliza *order.setState*), tiene un pequeño inconveniente. La operación *finishPreparation* está coordinando las dos llamadas a *removeLines* y a *setState* sobre el mismo objeto *this.order*. Por **patrón Experto** tendría más sentido que fuera el propio objeto *this.order* quien coordinara estas dos llamadas; por ejemplo, que tuviera en cuenta que siempre que invocamos *order.setState(new PreparedOrderState(order))* debemos invocar también *order.removeLines(cancelledLines)*. Para ello reescribimos *finishPreparation* haciendo que invoque una única operación (nueva) de *this.order*:

```

@Override
public void finishPreparation(List<String> cancelledLines) {
    this.order.setPreparationFinished(cancelledLines);
}

```

La nueva operación que necesitamos en la clase *Order* queda así:

```

public class Order {
    ...
}

```

```

void setPreparationFinished(List<String> cancelledLines) {
    removeLines(cancelledLines);
    setState(new PreparedOrderState(this));
}
private void removeLines(List<String> lines) {
    List<OrderLine> resultingOrderLines = new LinkedList<>();
    for(OrderLine ol : orderLines){
        if(!ol.isForProductIn(lines)){
            resultingOrderLines.add(ol);
        }
    }
    this.orderLines = resultingOrderLines;
}
}

```

Para saber si una línea de pedido pertenece o no a la lista de las que tenemos que eliminar no hemos podido utilizar tal cual la operación *isForProduct* pero hemos creado una que sigue la misma idea (también guiados por el **patrón Experto**):

```

public class OrderLine {
    ...
    boolean isForProductIn(List<String> productNames) {
        return productNames.contains(product.getName());
    }
}

```

Nota: En Java sería mucho más natural (y eficiente) que *productNames* fuera un *Set* y no una *List*. Pero al enunciado ya era una *List* (por razones docentes, para no introducir innecesariamente detalles técnicos en el enunciado). Este sería más un detalle de implementación que de diseño y, por tanto, no nos vamos a fijar en ello (más allá de esta nota).

Por último, *PreparedOrderState* representa el estado de los pedidos ya preparadas. Como ninguna de las dos operaciones es válida, en este caso, no es necesario que redefina ninguna operación.

```

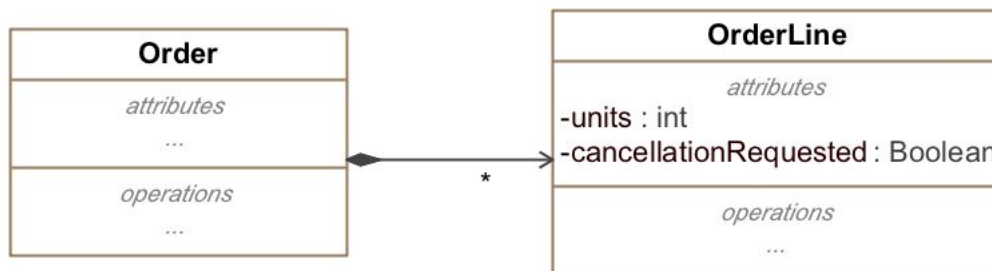
public class PreparedOrderState extends OrderState {
}

```

Ejercicio 3 (30%)

Centrémonos ahora en la propia cancelación de líneas de pedido. Partimos de una solución parcial al problema, en la que tenemos las clases del dominio ya adaptadas parcialmente.

Como hemos dicho, cuando un usuario pida cancelar una línea de pedido para un pedido que no está ni preparado ni en preparación, la cancelación será inmediata. Hemos decidido que en tal caso el sistema borrará directamente aquellas líneas canceladas y no quedará constancia de que se pidieron y cancelaron. Pero si el usuario solicita una cancelación de una línea del pedido de un pedido en preparación, el sistema sólo tiene que registrar la solicitud de cancelación. Hemos decidido modelar como un atributo en la clase `OrderLine`:



Al finalizar la preparación del pedido, si había líneas con solicitud de cancelación que no se han podido cancelar, se dejarán tal cual. Pero si se han podido cancelar, sencillamente se eliminarán.

Supongamos, también, que hemos ampliado la clase `Orders` con las siguientes operaciones:

```
// retorna la comanda d'un client a partir del número de comanda
getOrder(orderId:String): Order
```

```
// desa els canvis a una comanda
updateOrder(order: Order)
```

Hemos decidido que habrá una nueva clase controladora `UpdateOrder` para este caso de uso:

```
public class UpdateOrder {

    private final Orders orders;

    public UpdateOrder(Orders orders) {
        this.orders = orders;
    }
}
```

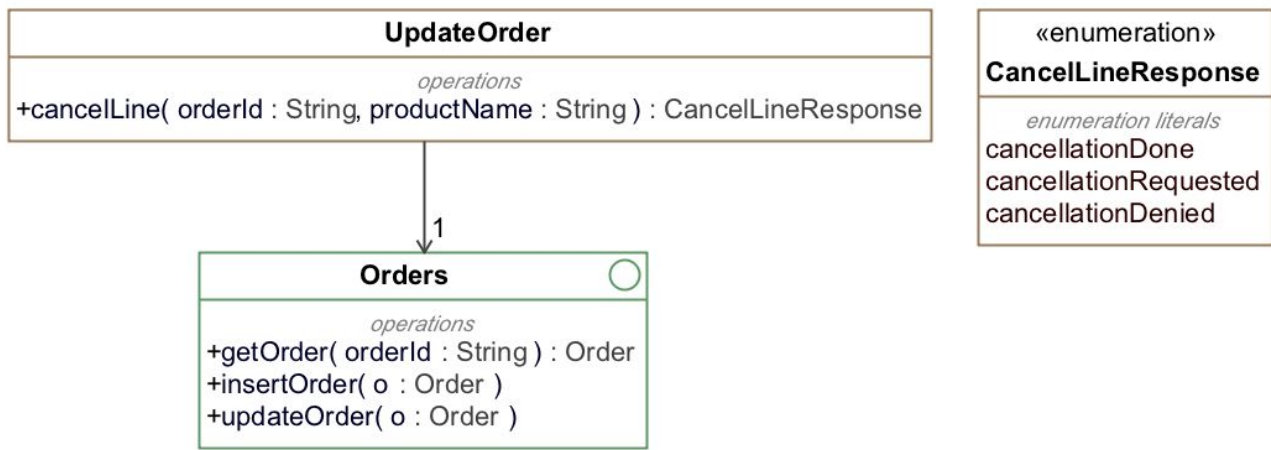
```

}

public CancelLineResponse cancelLine(String orderId, String productName){
    // PENDIENTE: Diseñar
}
}

```

Aquí, *CancelLineResponse* es una enumeración. El diagrama de clases queda así:



- Diseña en pseudocódigo o en un diagrama de secuencias UML la operación *cancelLine(orderId:String, productName: String):CancelLineResponse* de la clase *UpdateOrder* teniendo en cuenta que:
 - Hay que asignar responsabilidades de forma apropiada
 - Hay que evitar la repetición de código (principio de No Repetición o DRY)
- Justifica tu solución razonando sobre cómo has asignado las diferentes responsabilidades a las clases.

Notas:

- Puedes ignorar la gestión de errores, en particular el caso de que se pida cancelar una línea para un producto que no está. O, si lo prefieres, puedes considerar que, en tal caso, se devuelve *cancellationDenied*.

- La solución que propongamos debe ser coherente con el diseño inicial presentado en este enunciado y con tu solución de los ejercicios anteriores.

Solución:

En este ejercicio hacemos un diseño completo de una operación desde el controlador *UpdateOrder* hasta las clases de dominio.

Empezamos por el controlador:

```
public class UpdateOrder {
    private final Orders orders;
    ...
    public CancellineResponse cancelline(String orderId, String productName){
        Order o = orders.getOrder(orderId);
        CancellineResponse result = o.cancelline(productName);
        orders.updateOrder(o);
        return result;
    }
}
```

El **patrón Experto** nos dice que el experto en cancelar una línea de pedido es el objeto que representa el pedido. Por lo tanto, lo primero que queremos hacer desde el controlador es recuperar este objeto para delegarle esta responsabilidad. Como ya vimos, recuperamos el objeto mediante la abstracción *Orders*, que representa el conjunto de pedidos del sistema. La operación de *Orders* que usamos eventualmente se implementará como una consulta a la base de datos.

Una vez localizado el objeto que representa el pedido, invocamos sobre el mismo una nueva operación *cancelline* (que diseñaremos a continuación), e informamos a *Orders* de que hemos modificado el objeto, de tal manera que la implementación pueda guardar los cambios en la base de datos .

A partir de aquí, estamos en el terreno de las clases de dominio. Veamos *Order* y su *estado*:

```
public class Order {
    CancellineResponse cancelline(String productName) {
        return state.cancelline(productName);
    }
}
```

```

    }
}

public abstract class OrderState {
    CancellLineResponse cancellLine(String productName){
        return CancellLineResponse.cancellationDenied;
    }
}

```

Como en el ejercicio anterior, hemos aplicado el **patrón Estado**, ya que el comportamiento de *cancellLine* depende del estado actual del pedido. También hemos decidido (algo arbitrariamente) que el comportamiento por defecto de *cancellLine* es considerar que el estado es incorrecto, de tal manera que sólo sea necesario redefinir la operación en aquellos estados en que tenga sentido. En este caso, sin embargo, un estado incorrecto implica que devolvemos *CancellLineResponse.cancellationDenied* en lugar de lanzar una excepción.

En el caso de los pedidos que no han iniciado su preparación:

```

public class PlacedOrderState extends OrderState {
    private final Order order;
    ...
    public CancellLineResponse cancellLine(String productName) {
        boolean removed = order.removeLine(productName);
        if(!removed) return CancellLineResponse.cancellationDenied;
        return CancellLineResponse.cancellationDone;
    }
}

```

Necesitamos borrar la línea del pedido y devolver *CancellLineResponse.cancellationDone*. Hemos asumido, también, que queremos devolver *CancellLineResponse.cancellationDenied* si el pedido no tenía ese producto. Según el **patrón Experto**, quien tiene que eliminar la línea es el objeto que representa el pedido, así que delegamos en este objeto, en una nueva operación que diseñaremos a continuación, la eliminación de la línea.

Hubiéramos podido reutilizar la operación que elimina varias líneas de pedido, pero nos ha parecido más claro crear una nueva operación para el caso en que sólo tenemos una línea que eliminar. Hemos asumido, también, que la operación nos devolverá si se ha encontrado o no la línea a eliminar y que en ninguno de los dos casos lanzará excepción, por lo tanto.

La nueva operación en *Order* es:

```
public class Order {  
    ...  
    boolean removeLine(String productName) {  
        OrderLine ol = findOrderLine(productName);  
        if(ol == null) return false;  
        orderLines.remove(ol);  
        return true;  
    }  
}
```

Hemos implementado la nueva operación desde 0. También hubiéramos podido reutilizar internamente la operación *removeOrderLines*, pero la hubiéramos tenido que modificar para que devolviera el número de líneas que ha encontrado o, al menos, si ha encontrado alguna o no, para así poder devolver si hemos encontrado o no la línea que se quería borrar.

Si el pedido está en preparación:

```
public class InPreparationOrderState extends OrderState {  
    ...  
    public CancelLineResponse cancelLine(String productName) {  
        boolean found = order.setLineCancellationRequested(productName);  
        if(!found) return CancelLineResponse.cancellationDenied;  
        return CancelLineResponse.cancellationRequested;  
    }  
}
```

En este caso hay que marcar la línea con una solicitud de cancelación. Modelaremos esto como un simple booleano en la línea. De forma similar al caso anterior, por el **patrón Experto**, hemos decidido que sea *this.order* quien se encargue de marcar la línea con esta solicitud y es el estado quien devuelve uno u otro resultado en función de si se ha encontrado la línea o no.

La nueva operación *setCancellationRequested* de *Order* queda así:

```
public class Order {  
    ...  
    boolean setLineCancellationRequested(String productName) {  
        OrderLine ol = findOrderLine(productName);  
        if(ol == null) return false;  
    }  
}
```

```

        ol.setCancellationRequested();
        return true;
    }
}

```

El pedido debe localizar la línea en cuestión (reutilizando la operación que ya teníamos) y, por **patrón Experto**, delegar en la propia línea el hecho de marcarla como solicitada por cancelación.

En *OrderLine* el atributo *cancellationRequested* se inicializa a *false* al crear la línea y el método *setCancellationRequested* sólo lo pone a *true*:

```

public class OrderLine {
    ...
    private boolean cancellationRequested = false;
    void setCancellationRequested() {
        this.cancellationRequested = true;
    }
}

```

De nuevo, se trata de un método de negocio diseñado con la guía del **patrón Experto**, no de un *setter*. De hecho, no tiene sentido que exista una manera de cambiar *cancellationRequested* a *false*, ya que una vez solicitada la cancelación, por lo poco que tenemos del enunciado, no hay manera de deshacer la petición de cancelación. Si la hubiese, ya diseñaríamos esa otra funcionalidad.

Gestión de productos

Nos centramos ahora en la gestión de productos del catálogo. Un producto, hasta ahora, tiene un nombre que lo identifica y un precio con histórico de precios. Ahora nos piden, pero, una funcionalidad de cambio de nombre de un producto para gestionar el catálogo y poder hacer correcciones y actualizaciones. Para ello, un primer cambio que habrá que hacer será identificar los productos mediante un identificador diferente del nombre.

Pero, además, se quiere que cada producto tenga asociado un histórico de los cambios de nombre o de precio por los que ha pasado. Esto nos permitirá ver qué cambios ha tenido el producto a lo

largo de la vida, incluyendo quien hizo el cambio, en qué fecha lo hizo y qué cambio fue (de nombre, de precio, etc). Además, se quiere poder deshacer el último de estos cambios.

Ejercicio 4 (20%)

En este ejercicio nos centraremos sólo en los cambios de nombre y en el modelo del dominio. Para ello partimos de un diseño inicial de las operaciones necesarias:

Product	
<i>attributes</i>	
-id : ID	
-name : String	
<i>operations</i>	
+changeName(userEmail : String, at : DateTime, newName : String)	
+undoLastChange()	
...	

La operación *changeName(userEmail:String, at:DateTime, newName:String)* debe cambiar el nombre del producto pero también registrar el cambio para futuras consultas. Los parámetros *userEmail* y *at* indican qué empleado ha hecho el cambio y en qué momento lo ha hecho, ya que es necesario guardar esta información como parte del cambio de nombre que se guardará en el histórico de cambios de nombre del producto.

La operación *undoLastChange()* debe deshacer el último cambio, dejando el nombre del producto que había antes del último cambio de nombre. Fíjate que debe ser posible invocar varias veces *undoLastChange()* y, por tanto, deshacer varios cambios.

- Diseña en pseudocódigo o en un diagrama de secuencias UML las operaciones *changeName(userEmail:String, at:DateTime, newName:String)* y *undoLastChange()* de la clase *Product* teniendo en cuenta que:
 - Hay que asignar responsabilidades de forma apropiada.
 - Hay que evitar la repetición de código (principio de No Repetición o DRY).
- Diseña en pseudocódigo o en un diagrama de secuencias UML una nueva operación que permita consultar todos los cambios de nombre que ha tenido el producto.

- c) Justifica tu solución razonando sobre cómo has asignado las diferentes responsabilidades a las clases.

Nota:

- La solución que propongamos debe ser coherente con el diseño inicial presentado en este enunciado y con tu solución de los ejercicios anteriores.

Solución:

Hay muchas posibles soluciones con muchos enfoques posibles para resolver este ejercicio.

Pero la combinación de querer tener un histórico de cambios de nombre de un producto y poder deshacer el último cambio incluye dos de los problemas que el **patrón Orden** (*Command*) nos permite resolver. Veremos una solución basada en este patrón.

Según el patrón, cada instancia de una orden (*command* en inglés) representa una petición, tiene todos los datos de la petición y tiene una operación para, efectivamente, llevar a cabo esta petición. Como queremos poder deshacer operaciones realizadas, podemos añadir a las órdenes una operación para deshacerlas.

Hemos diseñado una abstracción con todo lo que pensamos que será común a las órdenes asociadas a productos:

```
public abstract class ProductCommand {  
    abstract void execute(Product product);  
    abstract void undo(Product product);  
}
```

En el caso particular del cambio de nombre tenemos:

```
public class ChangeNameProductCommand extends ProductCommand {  
    private final String userEmail;  
    private final Instant at;  
    private final String newName;  
    private final String oldName;  
  
    public ChangeNameProductCommand(Product currentProduct, String userEmail, Instant at,  
    String newName) {
```

```

        this userEmail = userEmail;
        this at = at;
        this newName = newName;
        this oldName = currentProduct.getName();
    }
    @Override
    void execute(Product product) {
        product.setName(newName);
    }

    @Override
    void undo(Product product) {
        product.setName(oldName);
    }
}

```

Lo primero que hay que explicar es que *ChangeNameProductCommand* tiene los datos de la petición (*userEmail*, *at* y *newName*) pero también los datos que necesitará para deshacer el cambio, en este caso, *oldName*.

Tanto la ejecución como la operación de deshacer de esta orden lo que hacen es cambiar el nombre del producto. Por **patrón Experto**, es el objeto que representa el producto el que debe poder hacer cambios de nombre de un producto. Por lo tanto, hemos añadido un atributo a *ChangeNameProductCommand* que represente el objeto del que se ha pedido el cambio de nombre y hemos delegado en este objeto el cambio de nombre (tanto para ejecutar el cambio de nombre inicial como para deshacerlo si es el caso). En *Product*, la operación *setName* sólo cambia el atributo *name* del producto y, por tanto, no se muestra (aún, en el código que hemos visto hasta ahora).

El enunciado ya indicaba que las operaciones *changeName* y *undoLastChange* pertenecen a la clase *Product* y, de hecho, tiene todo el sentido del mundo que estén allí desde el punto de vista del **patrón Experto** (aunque internamente usamos el **patrón Command**).

Veamos cómo queda:

```

public class Product {
    private final List<ProductCommand> commands = new LinkedList<>();
    ...
    void changeName(String userEmail, Instant at, String newName){

```

```

        this.execute(new ChangeNameProductCommand(this, userEmail, at, newName));
    }
    private void execute(ProductCommand cmd){
        cmd.execute(this);
        commands.add(cmd);
    }
    void undoLastChange(){
        ProductCommand cmd = commands.remove(commands.size() - 1);
        cmd.undo(this);
    }
    public List<ProductCommand> getCommands() {
        return commands;
    }
    void setName(String newName) {
        this.name = newName;
    }
}

```

Como hemos de poder deshacer el último cambio de un producto determinado (no el último cambio en el conjunto del sistema, como hacen Ct+Z o Cmd+Z), hemos asociado al producto una lista de las órdenes que se han ejecutado. Esta lista es inicialmente vacía.

La operación *changeName* crea una orden que represente el cambio de nombre (una instancia de *ChangeNameProductCommand*), la ejecuta y la añade a la lista. Teniendo en cuenta que "ejecutar una orden y añadirla a la lista" será genérico para cualquier tipo de orden de producto (por ejemplo para el cambio de precio), hemos creado una operación específica para ello (*execute*).

La operación *undoLastChange* extrae de la lista el último elemento añadido (en Java es un poco complicado, *commands.remove(commands.size() - 1)*); es decir, recupera la última orden de la lista y la elimina de dicha lista. Una vez la ha obtenido, ejecuta la operación *undo* de la orden.

Por último, *getCommands* es la consulta de todas las órdenes y es tan sencilla como devolver el conjunto de órdenes que tenemos. Y *setName* es la operación de cambio de nombre que sólo utiliza la orden *ChangeNameProductCommand*.

Nota sobre una posible variante

Otra forma de resolver el problema hubiera sido modelar el conjunto de nombres que el producto ha tenido a lo largo del tiempo como una **asociación histórica**. Esta solución permitiría resolver también la funcionalidad de deshacer y, en este sentido, no sería una mala solución.

El inconveniente, sin embargo, sería que la solución no resolvería cómo guardar otros cambios históricos del producto (como el cambio de precio) ni cómo deshacerlos. Dado que ya se enunciaba que queremos tener la misma funcionalidad por los cambios de precio, vale la pena que lo pensemos detenidamente. Así, si tenemos una asociación histórica para los nombres del producto a lo largo del tiempo, cuando queramos añadir los cambios de precio, habrá que reimplementar la funcionalidad de deshacer para el caso particular de los precios. Esto acabará suponiendo una violación del **principio de No Repetición**.

Es cierto, pero, que para el caso de los precios ya tenemos una asociación histórica. Por lo tanto, dos alternativas válidas (y no son las únicas) para implementar el futuro *undoLastChange* capaz de deshacer cambios de precios serían:

- Utilizar el enfoque de dos asociaciones históricas e implementar *undoLastChange* ad-hoc para cada una de ellas.
- Utilizar el enfoque de histórico de órdenes y reescribir la consulta del precio del producto en un momento pasado mediante la historia de órdenes del producto.

En esta solución hemos asumido la segunda variante pero no hemos reescrito la consulta del precio histórico de un producto para no alargar demasiado la solución y porque el enunciado del ejercicio 5 asumía que continuábamos usando dicha asociación histórica.

Ejercicio 5 (10%)

Supón que ahora nos piden que un producto pueda dejar de tener precio durante un periodo de tiempo, por ejemplo, porque está desactivado y no se vende. Para soportar esta funcionalidad tendremos que modificar la asociación histórica que ya teníamos para incluir el período de validez de cada precio (y no sólo la fecha y hora de inicio de validez). Pero no nos queremos fijar en eso.

Lo que nos preocupa es que hay varios puntos donde usamos el precio de un producto para hacer cálculos ... y no están preparados para soportar productos sin precio. Por ejemplo, esta versión revisada del cálculo del precio de un pedido:

```
public class Order {
    ...
    private final List<OrderLine> orderLines;
    private Money getTotalPrice() {
        Money result = Money.zero;
        for (OrderLine ol : orderLines) {
            result = result.sum(ol.getPrice(this.orderedAt));
        }
        return result;
    }
}

public class OrderLine {
    private final Product product;
    private int units;
    public Money getPrice(Instant orderCreationInstant) {
        Money productPrice = product.getPrice(orderCreationInstant);
        return productPrice.times(units);
    }
}
```

Podemos ver que se utiliza una clase *Money* para representar los precios y que esta clase tiene operaciones como *times* o *sum*:

```
public class Money {
    private final int cents;
    private Money(int cents) {
        this.cents = cents;
    }
    public static Money zero = Money.fromCents(0);
    public static Money fromCents(int cents){
        return new Money(cents);
    }
    public Money sum(Money b) {
        return fromCents(this.cents + b.cents);
    }
    public Money times(int units) {
```

```

        return fromCents(this.cents * units);
    }
}

```

Queremos poder tratar el caso de un producto que no tiene precio sin tener que modificar los cálculos ya disponibles en *Order* y *OrderLine*. Así el diseño queda más sencillo. Para ello, queremos que `product.getPrice(orderCreationInstant)` retorne algo que podamos utilizar normalmente, como cualquier instancia de *Money*, pero que represente el hecho de que no conocemos el precio de ese producto y, por tanto, que no conocemos el de la línea o el del pedido donde pueda estar. Aunque esto sería un error, preferimos ver que un pedido tiene un precio desconocido que no recibir la temible *NullPointerException* de Java o equivalente.

- Propón un diseño en pseudocódigo (o bien en diagramas de clases y de secuencias) que solucione este problema.
- Justifica tu diseño y, en caso de haber aplicado algún patrón de diseño, razona sobre por qué es adecuado para este problema.

Notas:

- NO hay que diseñar la nueva operación *getPrice* de *Product*, pero SÍ las operaciones que hacen la suma y multiplicación de precios en situaciones en las que quizás no conocemos uno de los precios.
- En caso de que sea necesario realizar cambios en las operaciones de *Order* y *OrderLine* que calculan los precios, indica cuáles son estas modificaciones.
- La solución que propongamos debe ser coherente con el diseño inicial presentado en este enunciado y con tu solución de los ejercicios anteriores.

Solución:

Por definición del propio patrón, el que nos permite tratar la ausencia de precio como si se tratara de un objeto que representa un precio es el **patrón Objeto nulo**.

Para aplicarlo es necesario crear una clase tal que una instancia de dicha clase represente un precio desconocido pero que se comporte como cualquier precio conocido; debe ser posible sumar

un precio (conocidos o no) a un precio desconocido y multiplicar un precio desconocido por un entero. Dicha clase debe ser, por tanto, una subclase de *Money* que implemente correctamente las operaciones *sum* y *times* para el caso en que desconocemos el precio de un producto.

Lo primero que hemos hecho es convertir *Money* en una interfaz de tal manera que la clase tal como la conocíamos sea sólo una de las dos posibles implementaciones:

```
public interface Money {
    static Money zero = Money.fromCents(0);
    static Money fromCents(int cents){
        return new MoneyInCents(cents);
    }
    Money sum(Money b);
    Money times(int units);
}
```

Como se puede ver hemos trasladado los miembros estáticos (*zero* y *fromCents*) a esta interfaz de tal manera que el cambio sea transparente a los usuarios de la antigua clase.

La operación *fromCents* ahora devuelve una clase concreta que implementa la interfaz. La hemos llamado *MoneyInCents*:

```
public class MoneyInCents implements Money{
    private final int cents;
    MoneyInCents(int cents) {
        this.cents = cents;
    }
    @Override
    public Money sum(Money b) {
        if(b instanceof MoneyInCents){
            return Money.fromCents(this.cents + ((MoneyInCents)b).cents);
        }
        return Money.unknown;
    }
    @Override
    public Money times(int units) {
        return Money.fromCents(this.cents * units);
    }
}
```

MoneyInCents contiene las mismas operaciones (no estáticas) y atributos que tenía la clase *Money* original. Lo único que cambia es que la operación *sum* recibe un objeto de tipo *Money*... del que no

sabemos si podemos consultar los céntimos, porque no tiene por qué ser una instancia de *MoneyInCents*. Lo que hacemos, pues, es comprobar si lo es. Si nos han pasado una instancia de *Money* que no lo es de *MoneyInCents* significa que intentamos sumar a una cantidad de dinero conocida una cantidad de dinero que no es conocida. En tal caso devolvemos el objeto que representa una cantidad desconocida. Para ello hemos creado una instancia (atributo estático) en *Money*:

```
public interface Money {
    ...
    static Money unknown = new UnkownMoney();
}
```

Por último, ahora sí, la clase *UnkownMoney* es la clase de la que será instancia nuestro objeto nulo:

```
public class UnkownMoney implements Money{
    @Override
    public Money sum(Money b) {
        return this;
    }
    @Override
    public Money times(int units) {
        return this;
    }
}
```

El resultado de cualquier operación sobre un precio desconocido es un precio desconocido y, como esta clase no tiene atributos, podemos utilizar siempre la misma instancia para representar el objeto nulo. Por eso ambas operaciones retornan simplemente *this*.

Nota final

Si os interesa la forma final de la solución implementada en Java la podéis encontrar publicada en Github. La rama "*solucio*" es la que contiene la solución que se ha descrito en este documento:

<https://github.com/agile-jordi/oo-design-tutorial/tree/solucio>

En caso de que encontréis algún defecto en dicho repositorio, las *pull requests* son bienvenidas! :-)